

Rackner FDI

Federal Document Intelligence — Frontend Reference & Integration Guide

Owner	Remy Tran — Full-Stack & System Design	Stack	Next.js 16 · React 19 · TypeScript 5 · Tailwind 4
Phase	Phase 1 — 4-week build, demo end of August	Tests	66 Playwright tests · desktop / tablet / mobile

What this app does

A capture/BD person searches live federal opportunities, decides fast whether one is worth pursuing, sees how much money is really behind it, and finds who to talk to. Every AI claim is cited back to the source text so a human can check it.

The 7/22 reframing replaced the old “upload a contract” flow. Contractors go looking for contracts — they do not arrive with one in hand. Cut in that pivot: upload-first intake, role picker, PII pre-upload scan, 3-day retention. Added: sign-in, lifecycle-plan profile, SAM.gov search, compatibility scoring, spend lookup, contact discovery.

The four questions the UI answers

- **What is out there?** — SAM.gov search + the recompute radar
- **Is it worth our time?** — compatibility score vs. the Opportunity Lifecycle plan, with cited obligations
- **How much money is behind it?** — USAspending.gov spend history
- **Who do I talk to?** — contact discovery, with Procurement Integrity guardrails

1 · Feature inventory

Every screen currently implemented, with the component that owns it.

1.1 · Sign-in — /login

Feature	Behavior	Owner file
Credential form	Work email + password, native validation, autoComplete hints.	app/login/page.tsx
Session handling	JWT stored in sessionStorage; clears on tab close.	lib/auth.ts
Redirect guard	Already signed in -> bounces to /. No session -> every page bounces here.	hooks/useRequireAuth.ts
Error surfacing	Backend rejection rendered inline under the form.	app/login/page.tsx
Security note	States hashing + isolated-container posture to the user.	app/login/page.tsx

1.2 · Home — /

Feature	Behavior	Owner file
Search	Free-text over title, agency, office, NAICS, set-aside, incumbent. Server-side in production.	app/page.tsx
Recompete radar	Timing presets incl. the 12–18 month capture window. Filters are query params, evaluated server-side.	components/TimingFilter.tsx
Suggested list	Ranked against the lifecycle plan; falls back to unscored when no plan is on file.	app/page.tsx
Opportunity card	Title, agency line, description, fit badge, timing/value/incumbent meta. Two variants: solicitation vs expiring award.	components/OpportunityCard.tsx
Fit badge	0–100 donut-free badge, banded green/amber/red. Shows '—' when unscored.	components/ScoreBadge.tsx
Lifecycle plan	Header chip opens the parsed fit profile; upload/replace the plan PDF.	components/LifecycleModal.tsx
App shell	Brand, plan chip, org, avatar menu with sign-out.	components/TopBar.tsx
States	Skeletons while loading, empty-result copy, inline errors. Stale results are replaced by skeletons when the filter changes.	app/page.tsx

1.3 · Opportunity analysis — /opportunity/[id]

The split-pane workspace: analysis on the left, source evidence on the right.

Feature	Behavior	Owner file
Compatibility score	Donut (0–100) + band, and the 8 weighted CAP factors each scored 1–5 with a rationale tooltip and a citation link.	components/CompatibilityPanel.tsx
Obligations	Grouped by time or type. Each shows the plain-English obligation, deadline chip, verbatim quote, citation, and a verified / not-verified marker.	components/ObligationsPanel.tsx
Click-to-cite	Clicking a quote, factor citation, or chat citation scrolls the source pane to that section and highlights the exact sentence.	components/SourcePane.tsx
Source pane	Parsed sections with headings and page numbers. Collapsible on desktop; a tab on mobile.	components/SourcePane.tsx
Spend history	USAspending bar chart by fiscal year, total obligated, incumbent + UEI, trend.	components/SpendPanel.tsx
Contact	Discovered email with verifier confidence, plus the Procurement Integrity flag when the solicitation is active.	components/ContactPanel.tsx
Assistant	Per-opportunity Q&A. Answers carry clickable citations wired to the same highlight path.	components/ChatPanel.tsx
Recompete state	No RFP yet -> obligations and source pane explain why, fit + spend + contact remain.	app/opportunity/[id]/page.tsx

1.4 · Cross-cutting

Concern	Implementation
Responsive	Split pane collapses below lg into an Analysis / Source tab toggle. Card grid goes single-column. Verified on desktop, iPad Mini, iPhone 13.
Auth	useRequireAuth guards every page; Bearer token attached to every backend call.
Design system	Navy #16324f, white, hairline #d7dee6 borders, wash #f5f7f9. Flat — no gradients, no glassmorphism. Tokens in lib/theme.ts.
Offline mode	With NEXT_PUBLIC_API_URL unset the whole app runs on lib/mock.ts — the demo fallback if the backend is unavailable.

2 · Architecture

Three layers, one seam. Components never call fetch directly; every backend call goes through lib/api.ts, and every shape is defined once in lib/types.ts.

```
frontend/src/  
  app/  
    login/page.tsx      sign-in  
    page.tsx            search + recompute radar + suggestions  
    opportunity/[id]/   the split-pane analysis workspace  
  components/          11 presentational components  
  hooks/               useRequireAuth  
  lib/  
    types.ts           THE LOCKED SCHEMA  
    api.ts             THE ONLY FILE THAT TALKS TO THE BACKEND  
    mock.ts            in-browser fake backend (demo fallback)  
    auth.ts            session token helpers  
    theme.ts           design tokens
```

The switch

One environment variable decides whether the app talks to the mock or the real backend. No component changes either way.

```
const BASE = process.env.NEXT_PUBLIC_API_URL;  
const USE MOCK = !BASE; // unset -> lib/mock.ts, set -> real HTTP
```

3 · The API contract

Ten routes. The backend must return these shapes exactly as defined in `lib/types.ts`. All requests except `/auth/login` carry **Authorization: Bearer <token>**.

Method + route	Request	Response type
POST <code>/auth/login</code>	{ email, password }	{ access_token }
GET <code>/profile</code>	— (user derived from JWT)	Profile
POST <code>/profile/lifecycle</code>	multipart: file (PDF)	LifecycleProfile
GET <code>/opportunities/search</code>	q, kinds, expiring_from, expiring_to	OpportunitySummary[]
GET <code>/opportunities/suggested</code>	kinds, expiring_from, expiring_to	OpportunitySummary[]
GET <code>/opportunities/{id}</code>	—	OpportunitySummary
GET <code>/opportunities/{id}/analysis</code>	—	Analysis
GET <code>/opportunities/{id}/document</code>	—	SourceDocument
GET <code>/opportunities/{id}/spend</code>	—	SpendSummary
GET <code>/opportunities/{id}/contact</code>	—	ContactResult
POST <code>/opportunities/{id}/chat</code>	{ question }	ChatAnswer

Field rules that are easy to get wrong

Rule	Why it matters
verbatim_quote must be an exact substring of the matching SourceSection.text	SourcePane does <code>text.indexOf(quote)</code> . Any whitespace or unicode difference and the highlight silently does nothing — the demo's grounding moment breaks with no error.
citation.section matches SourceSection.ref after stripping the section sign	"\$L.2" must resolve to ref "L.2", or the scroll target is not found.
FitFactor.score is 1–5, not 0–100; weight is 0–1 and weights sum to 1	The factor bars render score/5. A 0–100 score would overflow every bar.
Analysis.band is pursue conditional no_bid	Drives the donut color and verdict pill. Thresholds: ≥ 70 pursue, 50–69 conditional, < 50 no-bid.
expiry_date / months_to_expiry / current_award_value are non-null ONLY for kind = expiring_award	The card switches to the recompet layout on these fields. Non-null on a solicitation renders the wrong variant.
Unverified quotes must arrive with verified = false, not be dropped	The UI flags them. Silently dropping them hides a hallucination instead of surfacing it.

4 · Integrating frontend and backend

Step 1 — point the frontend at the backend

```
# frontend/.env (git-ignored)
NEXT_PUBLIC_API_URL=http://localhost:8000

# then
npm run dev
```

Leave it unset to run on the mock. That is also the demo-day rollback: unset the variable, redeploy, and the full UI still works.

Step 2 — backend prerequisites

Requirement	Detail
CORS	Allow the frontend origin (localhost:3000 in dev, the deployed host in production). This is the first thing that breaks.
Auth header	Accept Authorization: Bearer <jwt>. Login response field must be named access_token.
HTTPS	Required in production. Mixed content is blocked by the browser.
Content type	application/json on every route except /profile/lifecycle (multipart).

Step 3 — verify in this order

Each step exercises one more layer. Stop at the first failure.

#	Check	Confirms
1	Sign in with a real account	CORS, TLS, auth, token storage
2	Header chip shows the lifecycle plan	GET /profile + JWT round-trip
3	Suggested list renders with fit badges	Scoring joined to the opportunity list
4	Search returns filtered results	SAM.gov integration + server-side query
5	Open an opportunity; donut and factors render	Analysis shape, 1–5 scores, band
6	Click an obligation quote; source highlights	Citation grounding — the demo moment
7	Spend chart and contact card render	USAspending + email discovery
8	Ask the assistant a question	LLM round-trip + citation links

5 · Known gaps and risks

Honest list as of this build. Ranked by how likely each is to bite during integration.

Risk	Detail	Owner
HIGH	LLM latency. The mock answers in ~1s; a real gov-safe LLM reading a 100-page solicitation may take 30s or more. The client sets no fetch timeout, so a slow analysis shows the skeleton forever with no cancel. Needs either a 202 + poll endpoint or streaming — a contract change, so decide before integration week.	Remy + Aggrey
HIGH	Quote matching. Highlighting is an exact substring match. Any normalization difference between the extractor and the analysis output silently disables the demo's grounding moment.	Aggrey + Kaliza
MED	Raw error text reaches users. The client throws <code>`\${status}: \${body}`</code> , and the login page renders it directly — a failed login can display raw JSON. Needs an error-mapping layer.	Remy
MED	Silent secondary failures. Spend and contact use empty catch handlers; if those endpoints 500, the panels just do not appear, with no message.	Remy
GAP	Recompete radar has no backend. <code>kinds=expiring_award</code> with an expiry window needs a USAspending award index queryable by period-of-performance end date. Not in the current build plan.	Aggrey
GAP	Chat sends no history. <code>askChat(id, question)</code> has no prior turns, so follow-up questions will not work. Adding history changes the contract.	Remy + Kaliza
GAP	Deploy target unresolved. <code>amplify.yml</code> and the CI workflow still target Amplify, but the 7/22 meeting chose ECS/EC2. Blocks Week 1 containerization and Week 4 deploy.	Remy

Everything data-shaped is still mock

To be explicit for any reviewer: the loop, the rendering, and the interactions are real, but every value on screen today comes from `lib/mock.ts`. There is no live SAM.gov call, no USAspending call, and no LLM in the frontend. That is the seam working as designed — the data source swaps without the UI knowing.